



**POLITECNICO
DI TORINO**
Dipartimento di
Elettronica e
Telecomunicazioni



DIGITAL SYSTEMS ELECTRONICS

Academic year 2024-2025

LABORATORY NR. 4

DUE DATE: April 8, 2025

DELIVERY DATE: April 8, 2025

GROUP 20

CONTRIBUTIONS:

- Erfan Taheri
- Raman Jahandari
- Bardia Vafadar
- Ali Mansouri

The members of the group listed above declare under their own responsibility that no part of this document has been copied from other documents and that the associated code is original and has been developed expressly for the assigned project.

Part1: Gated SR Latch

As required by the lab assignment, the SR latch was implemented based on the circuit provided in the instructions. This design was described using structural modeling, with intermediate signals (R_g, S_g, Qa, and Qb). The keep attribute was applied to ensure Quartus preserved the intended gate-level structure during synthesis.

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4  ENTITY SRlatch IS
5  PORT ( Clk, R, S : IN STD_LOGIC;
6        Q : OUT STD_LOGIC);
7  END SRlatch;
8
9  ARCHITECTURE Structural OF SRlatch IS
10     SIGNAL R_g, S_g, Qa, Qb : STD_LOGIC;
11     ATTRIBUTE keep : boolean;
12     ATTRIBUTE keep OF R_g, S_g, Qa, Qb : SIGNAL IS true;
13 BEGIN
14     R_g <= R AND Clk;
15     S_g <= S AND Clk;
16     Qa <= NOT (R_g OR Qb);
17     Qb <= NOT (S_g OR Qa);
18     Q <= Qa;
19 END Structural;
20
```

The testbench was created to apply multiple test cases to the latch, such as reset, set, hold, and invalid inputs.

```
52  -- Stimulus process
53  stim_proc: PROCESS
54  BEGIN
55      -- Initial conditions
56      WAIT FOR 10 ns; -- Small delay to stabilize the circuit
57
58      -- Test Case 1: Reset the latch (R=1, S=0, Clk=1)
59      R <= '1';
60      S <= '0';
61      WAIT FOR clk_period;
62
63      -- Test Case 2: Set the latch (R=0, S=1, Clk=1)
64      R <= '0';
65      S <= '1';
66      WAIT FOR clk_period;
67
68      -- Test Case 3: Hold state (R=0, S=0, Clk=1)
69      R <= '0';
70      S <= '0';
71      WAIT FOR clk_period;
72
73      -- Test Case 4: Invalid state (R=1, S=1, Clk=1)
74      R <= '1';
75      S <= '1';
76      WAIT FOR clk_period;
77
78      -- Test Case 5: Return to a valid state
79      R <= '0';
80      S <= '1';
81      WAIT FOR clk_period;
82
83      -- Stop simulation
84      WAIT;
85  END PROCESS;
86
87  END behavior;
```

The waveform resulted from the simulation is the following:

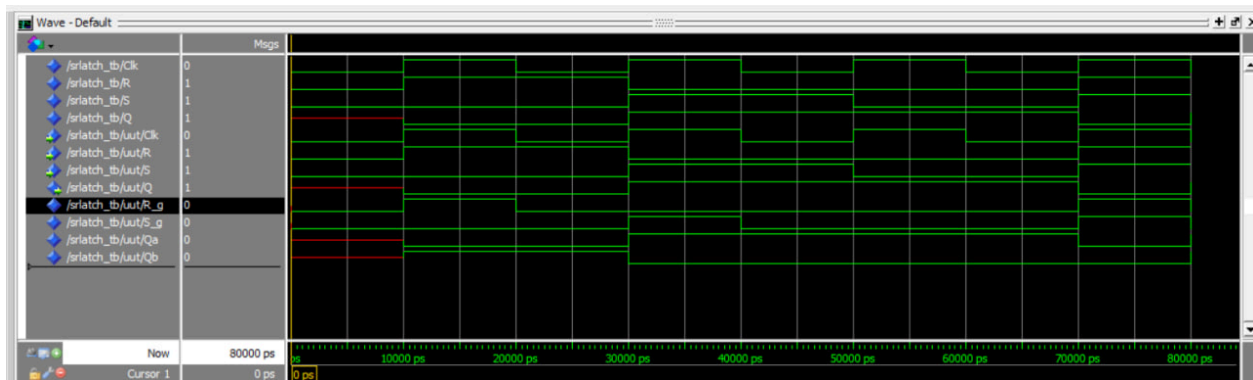


Figure 1 – Simulation waveform showing part1

After simulation, the design was compiled in Quartus Prime. The RTL and Technology Map Viewers were used to verify the gate-level implementation.

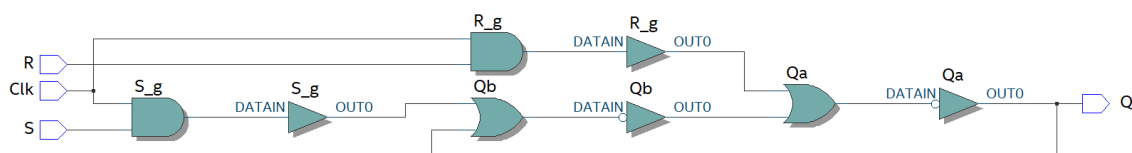


Figure 2– RTL Viewer: Gate-level structure of the SR Latch

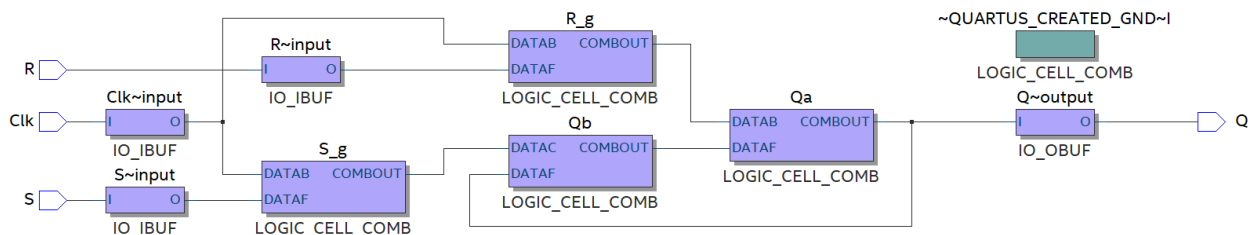


Figure 3 – Technology Viewer: Mapping of logic cells for SR Latch

The final implementation uses four LUTs, each corresponding to one of the internal signals. This matches the theoretical gated SR latch architecture and confirms the success of structural modeling.

Part2 : 16-bit synchronous counter

The objective was to design a **16-bit synchronous counter** using a structural approach, starting from a standard **T-type flip-flop**. This flip-flop was implemented with an **active-low asynchronous reset**, a **toggle input**, and a **rising-edge clock**. Once the T flip-flop was defined and tested, it was used as a building block to construct the full counter. Sixteen instances of the flip-flop were connected in a synchronous chain, where each flip-flop toggles only when all the previous ones are set forming a **carry chain**. The counter increments on each rising edge of KEY(0) when the **Enable** signal (SW(1)) is high, and it resets to zero when SW(0) is activated. The 16-bit output is split into four (**4-bit groups**) and displayed in hexadecimal on the **HEX0 to HEX3** 7-segment displays using a dedicated decoder module.

The first step was implementing the **T flip-flop** as a reusable module. It includes a clear input (active-low), a toggle control input, and a standard rising-edge-sensitive clock. The toggle behavior was achieved using an XOR operation between the current state and the toggle signal.

```

4  ENTITY Tflip_flop IS
5  PORT (
6      clk      : IN  std_logic; -- Clock input
7      clear    : IN  std_logic; -- Active-low asynchronous clear
8      toggle   : IN  std_logic; -- Toggle control
9      q_out    : OUT std_logic; -- Output
10 );
11 END Tflip_flop;
12
13 ARCHITECTURE behavior OF Tflip_flop IS
14     SIGNAL q_temp : std_logic := '0'; -- Internal state
15 BEGIN
16     PROCESS (clk, clear)
17     BEGIN
18         IF clear = '1' THEN
19             q_temp <= '0'; -- Asynchronous clear
20         ELSIF rising_edge(clk) THEN
21             q_temp <= toggle XOR q_temp; -- Toggle behavior
22         END IF;
23     END PROCESS;
24
25     q_out <= q_temp;
26 END behavior;
27

```

The **T flip-flop** was used to construct the **16-bit counter** in the top-level architecture. The first flip-flop toggles when Enable is high. Every subsequent flip-flop toggles only when the Enable signal and all previous flip-flops are high. This logic was implemented through a **carry_chain** signal that propagates through the stages. The counter also includes Reset and Enable signal handling via the switches, and Clock is controlled using KEY(0).

```

41 BEGIN
42     -- Control signal mapping
43     clk      <= KEY(0);
44     enable   <= SW(1);
45     reset_n  <= NOT SW(0); -- SW(0) is active-high; convert to active-low
46
47     -- First T flip-flop
48     tff_0 : Tflip_flop
49     PORT MAP (
50         clk      => clk,
51         clear    => reset_n,
52         toggle   => enable,
53         q_out    => counter_value(0)
54     );
55
56     -- Carry signal for second stage
57     carry_chain(0) <= enable AND counter_value(0);
58
59     -- Generate T flip-flops from bit 1 to 15
60     gen_counter : FOR i IN 1 TO 15 GENERATE
61         tff : Tflip_flop
62         PORT MAP (
63             clk      => clk,
64             clear    => reset_n,
65             toggle   => carry_chain(i - 1),
66             q_out    => counter_value(i)
67         );
68
69     -- Generate carry chain for i < 15
70     carry_gen : IF i < 15 GENERATE
71         carry_chain(i) <= carry_chain(i - 1) AND counter_value(i);
72     END GENERATE carry_gen;
73 END GENERATE gen_counter;
74
75     -- Output assignment
76     count_out <= counter_value;
77

```

The 16-bit counter is defined in the file “sixteen_bit_counter.vhd”. It receives the Clock signal through the button KEY(0) on the DE1 board, the Reset signal through switch SW(0) (active-high, internally

inverted), and the Enable control through switch SW(1). The current count is displayed in hexadecimal on the 7-segment displays HEX0 to HEX3. The display driver module was used in previous labs and is not described in detail here.

To display the counter output, the 16-bit binary value was split into four 4-bit sections and passed to a seven_seg_dec module, which converts each to a hexadecimal digit (0–F). This decoder was reused from a previous lab. We confirmed with the lab instructor that displaying the output in hexadecimal format was acceptable for this assignment. (should we show decoders code here again ?)

After completing the design, a **testbench** was developed to simulate the counter in ModelSim. The simulation included toggling the clock, enabling and disabling the counter, and applying reset signals at appropriate times. The counter incremented as expected, paused when disabled, and reset correctly when the reset signal was active. The waveform clearly showed the correct synchronous behavior and output transitions.

```

49  -- Stimulus Process
50  stim_proc: process
51  begin
52      -- Initial reset and disable
53      SW(0) <= '0'; -- Reset active (low)
54      SW(1) <= '0'; -- Enable off
55      wait for 2 * CLK_PERIOD;
56
57      -- Deassert reset
58      SW(0) <= '1';
59      wait for CLK_PERIOD;
60
61      -- Enable counting
62      SW(1) <= '1';
63      wait for 10 * CLK_PERIOD;
64
65      -- Disable counting
66      SW(1) <= '0';
67      wait for 5 * CLK_PERIOD;
68
69      -- Enable again
70      SW(1) <= '1';
71      wait for 10 * CLK_PERIOD;
72
73      -- End simulation
74      wait for 200 ns;
75      assert false report "Simulation finished successfully." severity failure;
76  end process stim_proc;
77
78  end architecture testbench;
79

```

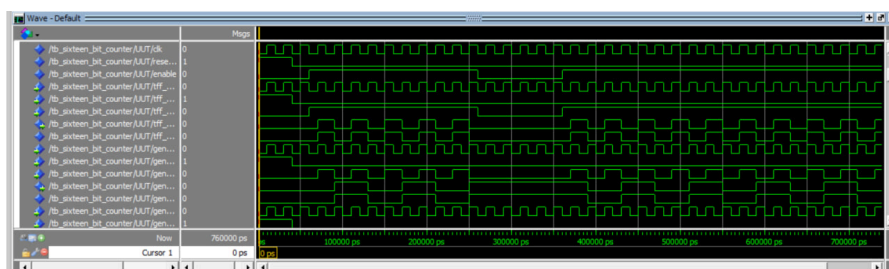
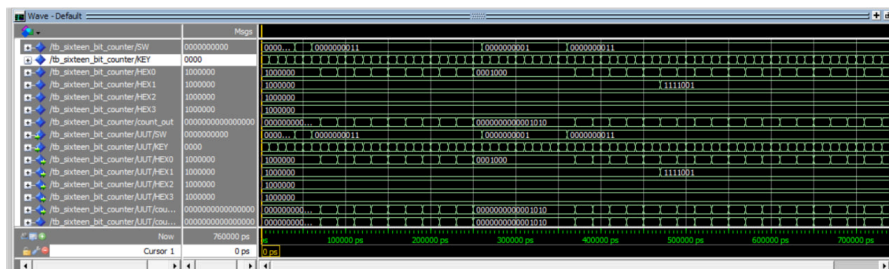


Figure 4– Simulation waveform showing part2

The design was synthesized in Quartus Prime, and the RTL Viewer confirmed a fully synchronous 16-bit structure with flip-flops connected via carry logic. The 7-segment display connections were also visible. Compared to the 4-bit schematic in Figure 4 of the lab, our implementation follows the same T flip-flop chaining logic but extends it to 16 bits with added display output, preserving the original design methodology.

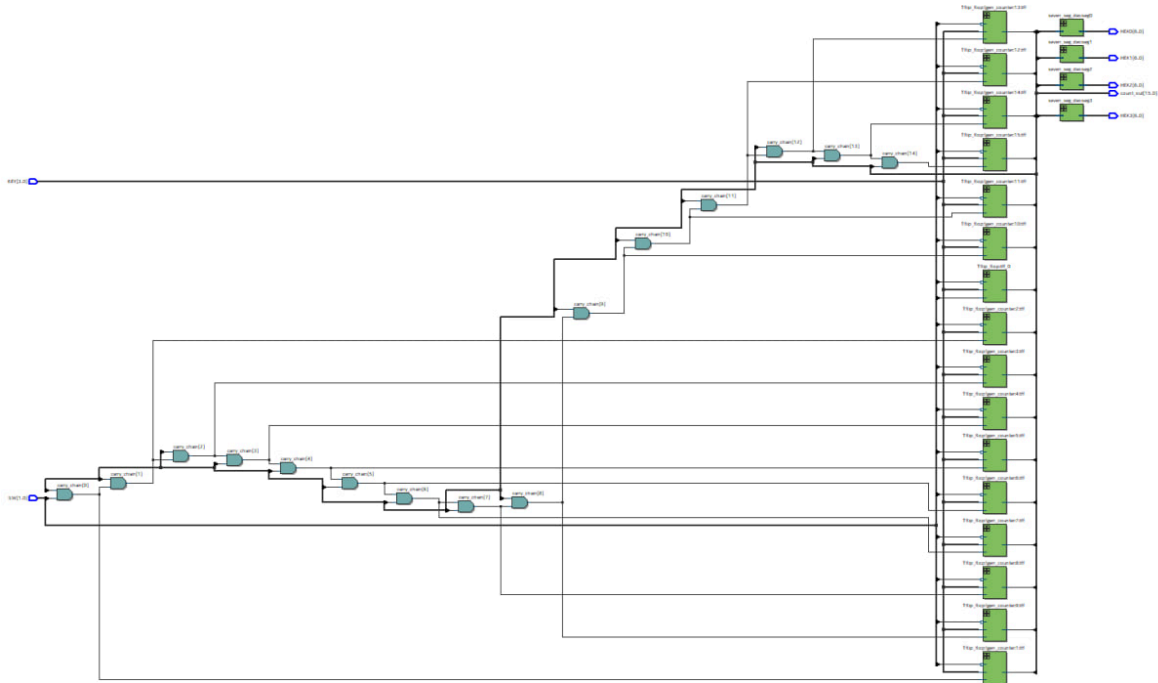


Figure 5 – RTL Viewer output for the 16-bit structural counter

In the RTL view, unlike the lab manual schematic where each T flip-flop's Q feeds directly into an AND gate, Quartus groups and reuses the output signals structurally. This slight visual difference is due to synthesis but doesn't affect the circuit's behavior.

Part3 : 16-bit synchronous counter version 2

In this part of the lab, the 16-bit synchronous counter was re-implemented using **behavioral VHDL**. The design, written in sixteen_bit_counterv2.vhd, uses a single process with a 16-bit unsigned signal (count_reg) to hold the counter value. On each rising edge of KEY(0), the counter increments by one when the Enable signal (SW(1)) is high and resets to zero when SW(0) is low. This version replaces the structural flip-flop instantiation and carry logic from Part 2 with simple control flow and arithmetic.

```

24 begin
25
26 -- Synchronous Process
27 process (KEY(0), SW(0))
28 begin
29     -- Active-low reset
30     if (SW(0) = '0') then
31         count_reg <= (others => '0');
32     elsif rising_edge(KEY(0)) then
33         if SW(1) = '1' then
34             count_reg <= count_reg + 1;
35         end if;
36     end if;
37 end process;
38
39 -- Output assignment
40 count_out <= std_logic_vector(count_reg);
41
42

```

What's new in this part This version uses **behavioral constructs** such as `rising_edge`, `if/elsif`, and `count_reg + 1` to describe the counter logic within a single process. This approach eliminates manual flip-flop instantiation and carry chains, simplifying the design while maintaining identical functionality to the structural version. The counter's 16-bit output is split into four 4-bit groups and displayed in hexadecimal on HEX0 through HEX3 using the `seven_seg_dec` module, which was reused from previous parts.

The simulation, carried out using the testbench `sixteen_bit_counterv2_tb.vhd`, toggled the clock and applied reset and enable signals to verify correct behavior. The counter incremented, paused, and reset correctly as expected.

```

50  -- Applies reset (active low) and toggles enable
51  stim_proc: process
52  begin
53      -- Initially: apply reset (SW(0)='0') and disable counting (SW(1)='0')
54      SW(0) <= '0'; -- Reset asserted (active-low)
55      SW(1) <= '0'; -- Enable = 0
56      wait for 3 * CLK_PERIOD;
57
58      -- Release reset (SW(0)='1')
59      SW(0) <= '1';
60      wait for 3 * CLK_PERIOD;
61
62      -- Enable counting (SW(1)='1')
63      SW(1) <= '1';
64      wait for 10 * CLK_PERIOD;
65
66      -- Disable counting
67      SW(1) <= '0';
68      wait for 5 * CLK_PERIOD;
69
70      -- Re-enable counting
71      SW(1) <= '1';
72      wait for 10 * CLK_PERIOD;
73
74      -- Finish simulation
75      wait for 200 ns;
76      assert false report "Simulation complete." severity failure;
77  end process stim_proc;
78
79  end architecture test;
80

```

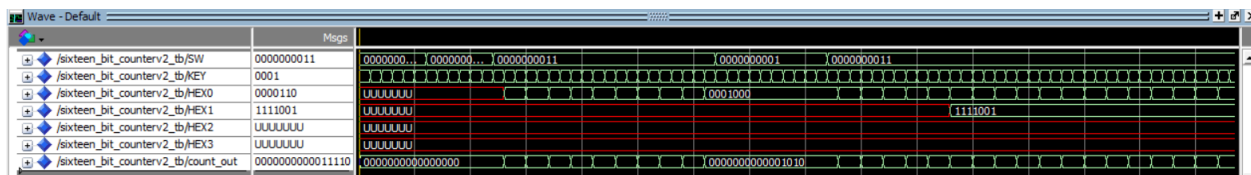


Figure 6 – Simulation waveform showing counting, reset, and enable behavior

The red lines in the waveform indicate undefined values ('U') for HEX1, HEX2, and HEX3 during the initial stages, likely due to uninitialized values before the reset completes. Once reset is released, the outputs are correctly defined.

After compiling, the behavioral design achieved a **maximum operating frequency (fmax)** of **460.19 MHz**, which is slightly lower than the **489.00 MHz** obtained with the structural version. This suggests that the manually optimized flip-flop structure in the structural design allows for slightly faster operation. The RTL Viewer shows a more abstract and compact implementation for the behavioral counter, while the structural version presents clearly defined flip-flop stages and carry logic, reflecting its gate-level construction.

Below is the RTL representation of this version as synthesized by Quartus:

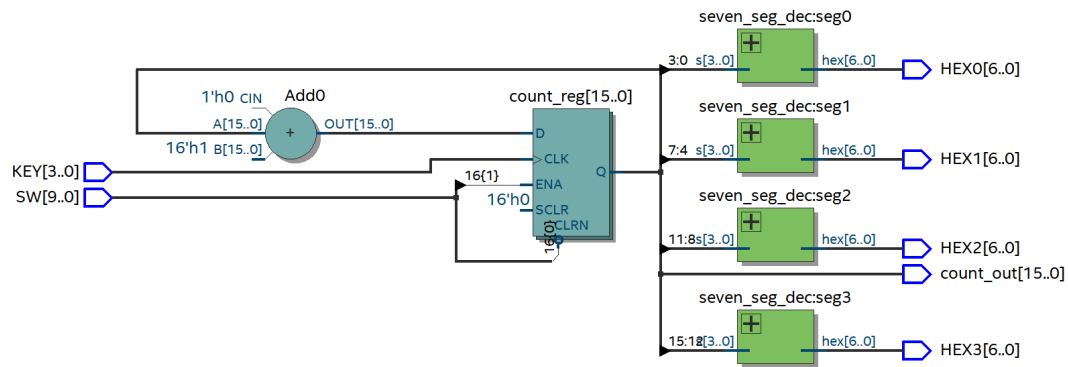


Figure 7 – RTL Viewer of behavioral 16-bit counter

Part4 : Flashing digits from 0 to 9

In this part, we use the 50 MHz clock as the main clock source for all components. To create a counter that ticks once per second, we need 26 flip-flops to count up to 50 million (which corresponds to one second). When this count is reached, the one_hz_tick signal goes high, enabling another group of four flip-flops that generate digits from 0 to 9. Moreover, the reset_n signal is pinned to switch zero (SW(0)) and the Enable signal is assigned to switch one (SW(1)).

The 4-bit output from these flip-flops is then used as the input for a 7-segment display. Figure 8 shows the structure of the circuit.

To make the counting cycle repeat, we included a VHDL process that resets all values after the 26-bit counter reaches one million. Here's the process we used in the VHDL code:

```

84 | process(CLOCK_50)
85 | begin
86 |     if rising_edge(CLOCK_50) then
87 |         if one_hz_tick = '1' then
88 |             reset_next <= '1';
89 |         else
90 |             reset_next <= '0';
91 |         end if;
92 |     end if;
93 | end process;

```

After reaching the one million count and setting the one_hz_tick signal to '1', we used four additional T-type flip-flops to count from 0 to 9.


```

-----
-- 4-bit Counter to count 0-9, incremented every 1 Hz
-----
-- First flip-flop
tff_0_digit : Tflip_flop
  PORT MAP (
    clk    => CLOCK_50,
    clear  => clear_digit,
    toggle => one_hz_tick,
    q_out  => counter_digit(0)
  );
carry_digit(0) <= one_hz_tick AND counter_digit(0);

-- Bits 1 to 3
gen_counter_digit : FOR i IN 1 TO 3 GENERATE
  tff : Tflip_flop
    PORT MAP (
      clk    => CLOCK_50,
      clear  => clear_digit,
      toggle => carry_digit(i - 1),
      q_out  => counter_digit(i)
    );
END GENERATE;

-- Carry logic for bits 1 and 2 only (since carry_digit has only 3 bits)
gen_carry_digit : FOR i IN 1 TO 2 GENERATE
  carry_digit(i) <= carry_digit(i - 1) AND counter_digit(i);
END GENERATE;

```

Finally, we reset the counter after reaching ten (1010) so that the digits repeat from 0 again :

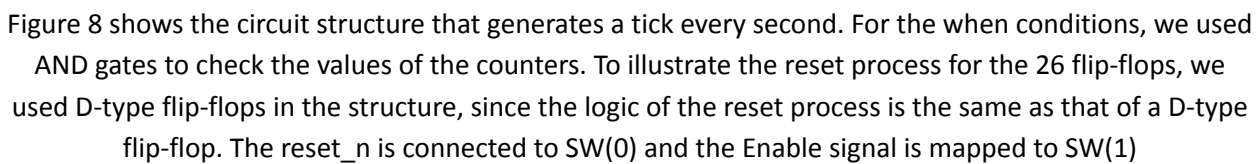
```

-----
-- Reset digit counter after 9
-----

clear_digit <= '1' WHEN reset_n = '1' OR (counter_digit = "1010" AND one_hz_tick = '1') ELSE '0';

-----
-- Display Digit on HEX0
-----
seg_display : seven_seg_dec
  PORT MAP (
    s    => counter_digit,
    hex  => HEX0
  );
END structural;

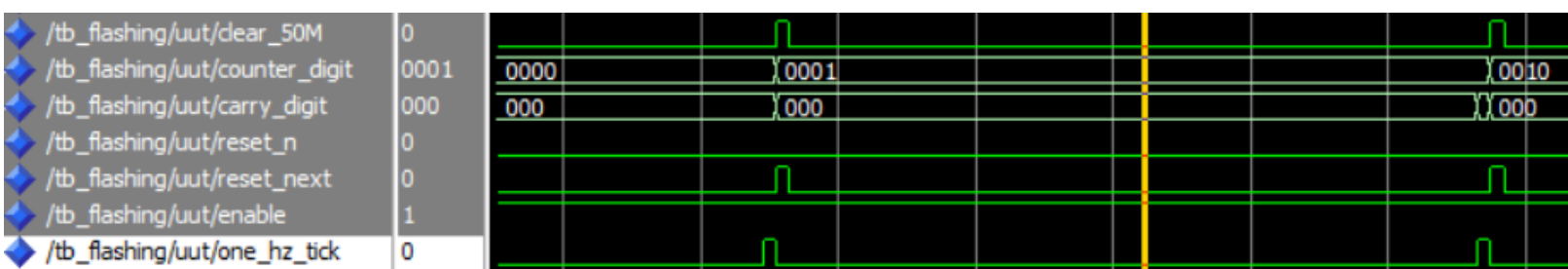
```



Below in Figure 9 is the resulting waveform from the simulation.



(a)



(b)

Figure 9. In part (a), the overall functionality of the circuit is shown. It confirms that the counting is working correctly, as the counter_digit values change exactly every 1000 ns (1 μ s). In part (b), we can see that for each transition, the one_hz_tick signal is triggered first, followed by the digit change. At the same time, clear_50M is activated to reset the 26 flip-flops so the counting can start over.

Part5 : Reaction timer

In this part, we built a reaction timer that measures how fast a user responds to a visual signal. After reset (KEY0), the system waits for a delay set by SW7–0 (0–255 ms). Once the delay passes, the LED (LED0) turns on and a 1 kHz counter starts. When the user presses KEY3, the LED turns off and the counter stops, showing the reaction time on HEX3–0. Pressing KEY0 resets the system. Figure 10 shows a circuit level view of the implementation based on the VHDL logic.

To convert the binary counter value to decimal, we used a BCD conversion component that translates a 16-bit binary number into four decimal digits. Internally, we used a signal called state with three main states: WAITING (default), COUNTING, and STOPPED. While the circuit structure uses binary encoding for these states (e.g., "00", "11"), the VHDL code defines them using a custom TYPE for readability and reliability.

Instead of explicitly instantiating flip-flops, the VHDL design uses counters and state transitions triggered on each clock cycle, simplifying the implementation and reducing the risk of errors. Below are code snapshots to demonstrate the logic used in the VHDL design:

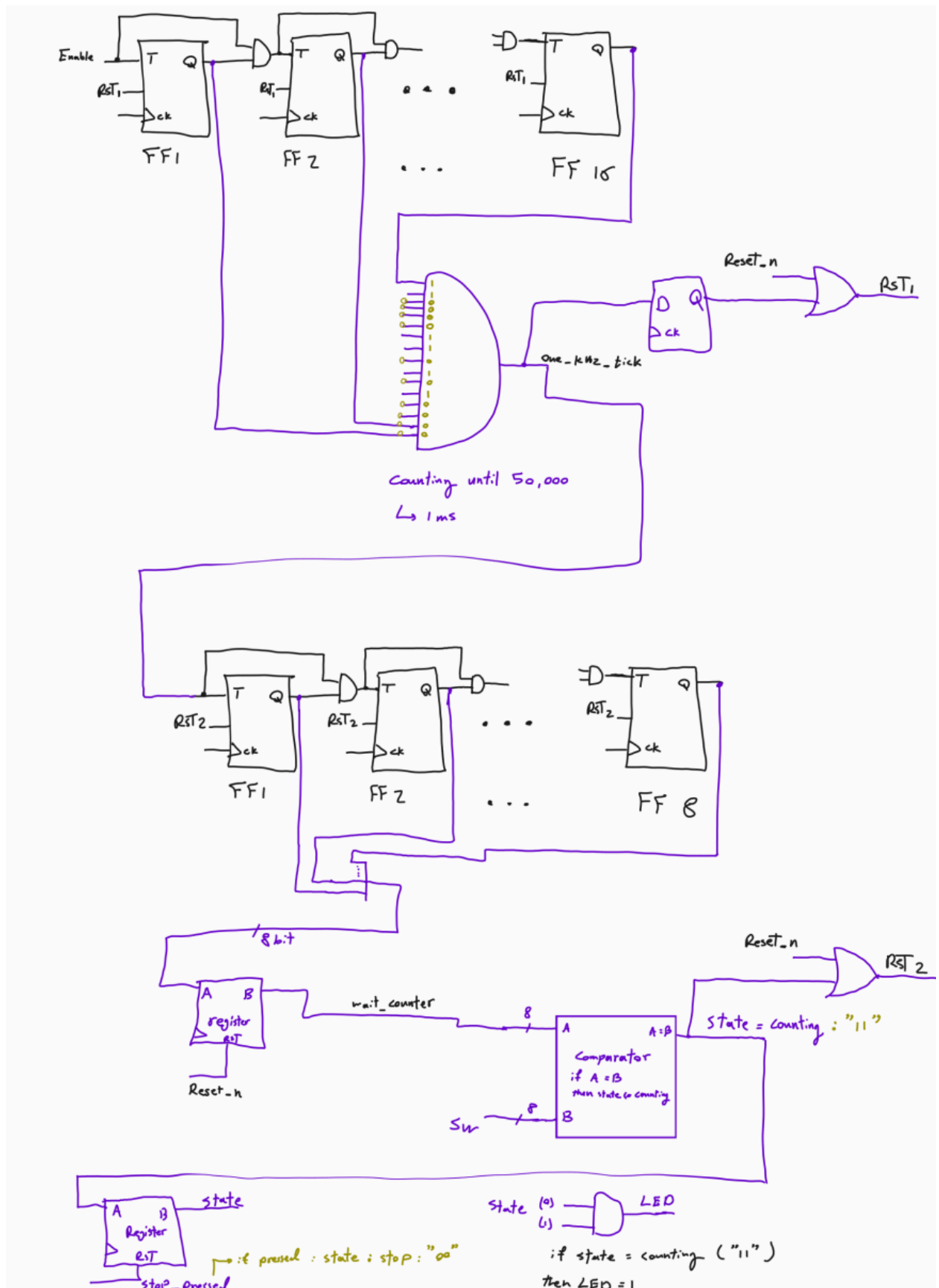
```
35 TYPE state_type IS (WAITING, COUNTING, STOPPED);
36 SIGNAL state : state_type := WAITING;
37
38 SIGNAL clk_div_count : unsigned(15 DOWNTO 0) := (others => '0');
39 SIGNAL one_khz_tick : std_logic := '0';
40
41 SIGNAL wait_counter : std_logic_vector(7 DOWNTO 0);
42 SIGNAL reaction_counter : std_logic_vector(15 DOWNTO 0);
43
44 SIGNAL reset_pressed : std_logic;
45 SIGNAL stop_pressed : std_logic;
46 SIGNAL led_on : std_logic := '0';
47
48 SIGNAL d0, d1, d2, d3 : std_logic_vector(3 DOWNTO 0);
49
50 BEGIN
51     reset_pressed <= NOT KEY0;
52     stop_pressed <= NOT KEY3;
53     LED <= led_on;
54
55     -- 1 kHz clock divider using process
56     PROCESS(CLOCK_50)
57     BEGIN
58         IF rising_edge(CLOCK_50) THEN
59             IF reset_pressed = '1' THEN
60                 clk_div_count <= (others => '0');
61                 one_khz_tick <= '0';
62             ELSIF clk_div_count = to_unsigned(49999, 16) THEN
63                 clk_div_count <= (others => '0');
64                 one_khz_tick <= '1';
65             ELSE
66                 clk_div_count <= clk_div_count + 1;
67                 one_khz_tick <= '0';
68             END IF;
69         END IF;
70     END PROCESS;
```

(a)

```
72 -- State machine process
73 PROCESS(CLOCK_50)
74 BEGIN
75     IF rising_edge(CLOCK_50) THEN
76         IF reset_pressed = '1' THEN
77             wait_counter <= (others => '0');
78             reaction_counter <= (others => '0');
79             led_on <= '0';
80             state <= WAITING;
81         ELSE
82             CASE state IS
83                 WHEN WAITING =>
84                     IF (one_khz_tick = '1' AND (unsigned(wait_counter) /= unsigned(SW))) THEN
85                         wait_counter <= std_logic_vector(unsigned(wait_counter) + 1);
86                     ELSIF unsigned(wait_counter) = unsigned(SW) THEN
87                         led_on <= '1';
88                         state <= COUNTING;
89                     END IF;
90
91                 WHEN COUNTING =>
92                     IF one_khz_tick = '1' THEN
93                         reaction_counter <= std_logic_vector(unsigned(reaction_counter) + 1);
94                     END IF;
95                     IF stop_pressed = '1' THEN
96                         led_on <= '0';
97                         state <= STOPPED;
98                     END IF;
99
100                 WHEN STOPPED =>
101                     -- Wait for reset to return to WAITING
102                     NULL;
103             END CASE;
104         END IF;
105     END IF;
106 END PROCESS;
```

(b)

In part (a), the STATE type and internal signals are defined, followed by the first counting process that increments at 50 MHz to reach 50,000 (1 ms). In part (b), default values are set when reset is pressed. The system then waits for the user defined delay, counts the reaction time, and switches to STOP state when the user presses the stop key. The reaction time is then converted from 16-bit binary to four decimal digits using a BCD converter and displayed on four 7-segment displays.



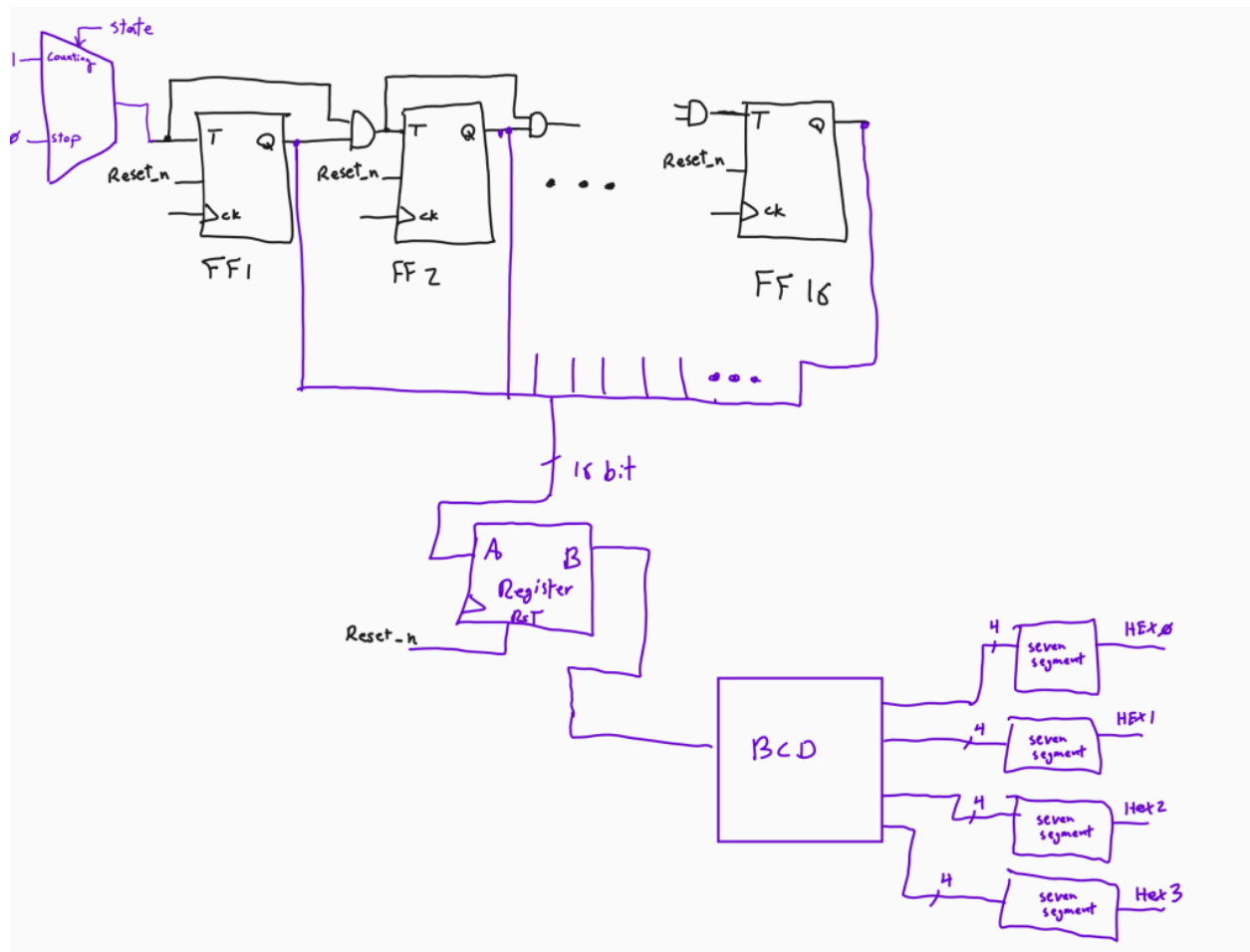


Figure 10. The circuit structure for the reaction timer. The Enable signal is the inverse of the reset_n signal, which is controlled by the user through KEY0. A 16-bit counter increments until it reaches 50,000, which corresponds to 1 millisecond. After each 1 ms tick, an 8-bit counter begins counting until it reaches the user defined delay set via the SW input. Once that value is reached, the 2-bit STATE signal changes to counting mode ("11"), enabling the third counter that tracks the user's reaction time. When the user presses KEY3 (i.e., stop_pressed = 1), the STATE returns to STOP mode ("00"), which is the default value stored in the STATE register. At this point, the counter stops and displays the user's reaction time in milliseconds.

Modelsim simulation :

For simulation and testing, we first define the 50 MHz clock in the testbench. Then, in a process, we apply a reset, set the waiting time to 3 ms using the switches, and simulate a reaction time of 5 ms. As shown in Figure 11, the circuit functions correctly, which can be verified by observing the STATE signal, reaction_counter, wait_counter, and the LED.

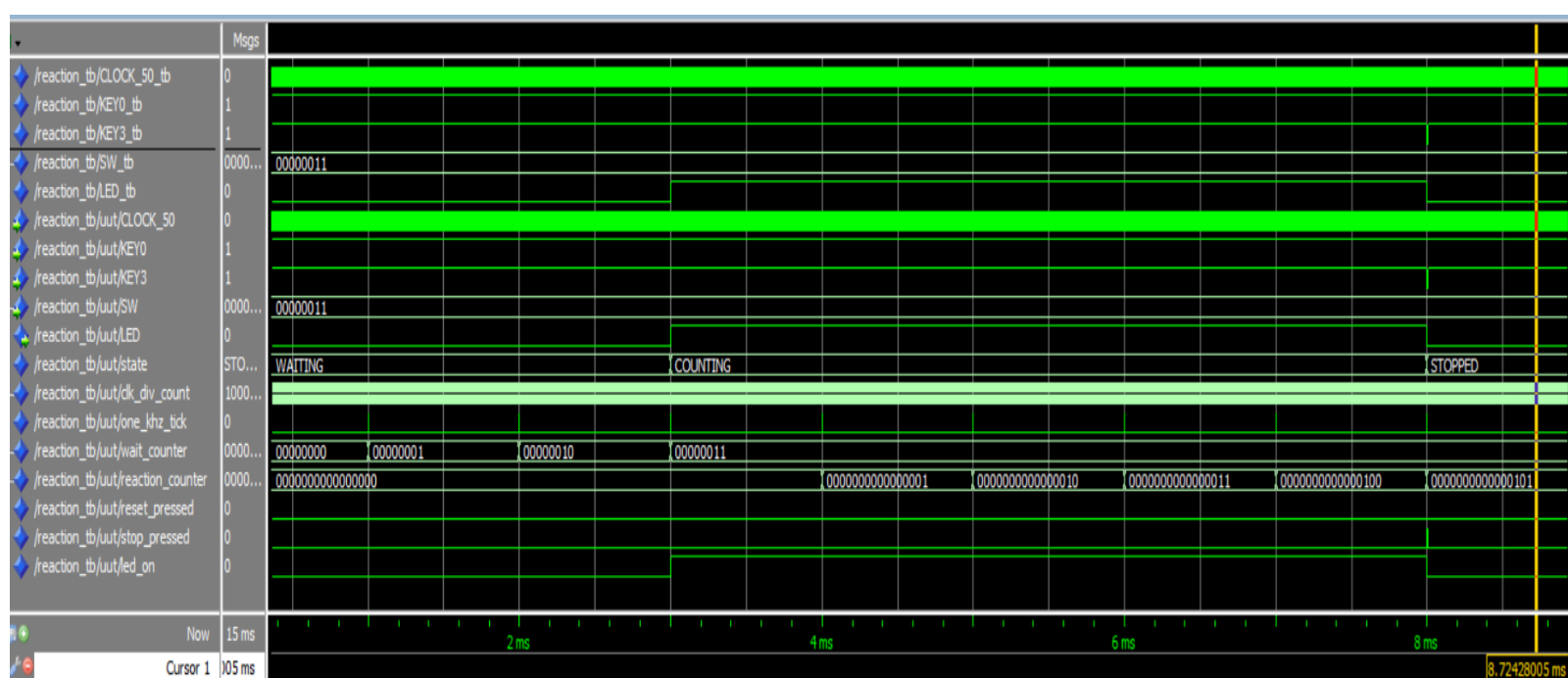


Figure 11. Simulation waveform of the reaction timer. The system starts in the WAITING state, and after 3 ms (as set by the switches), it transitions to COUNTING with the LED turned on. After 5 ms of reaction time, the stop button is pressed, switching the state to STOPPED, turning off the LED, and freezing the final reaction time on the counter.

Conclusion :

In Lab 4, we implemented and tested key sequential components in digital systems using VHDL. Starting with the gated SR latch, we examined its internal logic and verified it through simulation and synthesis. We then designed both structural and behavioral versions of a 16-bit synchronous counter, analyzing their differences in complexity and resource usage. In Part 4, we developed a digit flasher displaying 0–9 at 1 second intervals using clock division and modular counting.

Finally, in Part 5, we built a reaction timer using a finite state machine and BCD conversion to display the user's response time. All parts were tested through simulation in ModelSim and successfully synthesized and verified on the DE1 board, confirming correct hardware functionality.